HEXA: Narrow-Band Reduction and Banded Divide and Conquer for Dense Hermitian Eigenproblems on Distributed-Memory GPU Systems

Nikhil Kodali

Abstract—Computing all eigenpairs of dense Hermitian matrices, often in every iteration of an outer method, is a common bottleneck in semidefinite programming, quantum chemistry, nuclear physics, machine learning and electronic-structure calculations. On GPUs, the reduction to tridiagonal form is bound by memory bandwidth and, in distributed memory, by its many dependent collectives. Two-stage reductions avoid this but add bulge chasing and a second back-transformation of the eigenvectors. We present HEXA, which instead reduces the matrix in one stage to a narrow band and solves the band eigenproblem by divide and conquer, leaving one back-transformation. For wide bands, each reduction step factors its columns by CholeskyQR2 with Householder reconstruction. The divide and conquer applies the rank-one updates of the band in sequence, with a Cauchy fast multipole method built from batched matrix products for the eigenvector updates of large merges. The band width trades reduction steps against divide-and-conquer work, and in distributed memory HEXA selects it at run time from micro-benchmarks, together with the data distribution of the reduction. The same source code runs on NVIDIA, AMD and Intel GPUs in single and double precision, real and complex. In FP64 strong scaling on up to 64 nodes, HEXA is 4.9–10.0 times faster than ELPA's one-stage solver on GH200 nodes, 3.0–7.6 times on MI250X nodes and 3.8–6.9 times on Max 1550 nodes, and faster than cuSOLVERMp and DLA-Future at every point compared. On one GPU in FP64 it is 1.9–2.5 times faster than cuSOLVER, 2.6–7.0 times than rocSOLVER and 2.5–3.3 times than oneMKL, with eigenvalues that agree with those of the baselines.

Index Terms—Dense Hermitian eigenvalue problem, divide and conquer, band reduction, fast multipole method, GPU, distributed memory, performance portability.

1 Introduction

Computing all, or a large fraction, of the eigenpairs of a dense Hermitian matrix is a common step across computational science, often repeated in every iteration of an outer method. First-order solvers for semidefinite programs project onto the positive semidefinite cone by an eigendecomposition, which dominates the cost of each iteration for large problems [1], [2]. Gaussian-basis Hartree–Fock and Kohn–Sham codes diagonalize the Fock matrix in every self-consistent field (SCF) iteration, which becomes the rate-determining step beyond about 2000 basis functions [3], [4]. Nuclear density functional solvers diagonalize Hermitian Hartree–Fock–Bogoliubov Hamiltonians of order 10^5–10^6 at each self-consistent step [5], and second-order optimizers for neural networks, such as Shampoo, eigendecompose dense preconditioner blocks repeatedly during training [6], [7]. In many-body physics, the Bethe–Salpeter equation for excited states needs at least half of the eigenpairs of a dense Hermitian matrix [8], and full exact diagonalization is limited in practice to matrices of order about 50,000 [9]. Large sparse eigenproblems lead to the same step, because subspace iteration solves a dense projected eigenproblem, the Rayleigh–Ritz step, in each of its iterations [10]. In Kohn–Sham density functional theory (DFT) these iterations recur within every SCF iteration [11].

In electronic-structure calculations, this dense eigensolve becomes the bottleneck at large scale. The Rayleigh–Ritz step often dominates subspace iteration when many eigenvalues are needed [11], and the developers of ELSI, a unified interface to Kohn–Sham electronic-structure solvers, describe the Kohn–Sham eigenproblem as the bottleneck of DFT simulations for systems well beyond several thousand atoms [12]. The DFT-FE, FHI-aims, PARSEC and SPARC developers name the eigensolver as a limiting step on the way to exascale [13]. In DFT-FE on Summit, the dense diagonalization takes 34% and 56% of the SCF time for 61,502 and 114,674 electrons, although only 10–15% of the eigenpairs are computed [14]. In DGDFT the subspace eigenproblem takes the largest fraction of the SCF time on 55,296 cores [11] and is the main performance limit on Sunway TaihuLight [15]. Tight-binding calculations spend 90–95% of a typical run in diagonalization [16], and in plane-wave codes it stops scaling at around a hundred processors and then limits the calculation [17].

The subspace construction step of these methods, Chebyshev filtering or the Hamiltonian–multivector products of Davidson and conjugate-gradient solvers, has been accelerated on GPUs in DFT-FE [18], [19], SPARC [20], Quantum ESPRESSO [21] and the ChASE library [22], as have finite-element matrix–multivector products [23]. This acceleration

• *N. Kodali is with the Indian Institute of Science, Bengaluru, India. E-mail: nikhilkodali@iisc.ac.in*

shifts the cost further to the dense eigensolve. In the GPU port of Abinit the filtering step becomes negligible and the Rayleigh–Ritz step dominates [24]. In fully relativistic DFT-FE calculations on Aurora, mixed precision speeds up the Chebyshev filtering 2.85× for 78,988 electrons but the SCF iteration only 1.56×. The Rayleigh–Ritz step, whose eigenproblem is solved by ELPA, then takes 290 of 403 s and is limited by communication [25].

The direct solvers these codes rely on reduce the matrix to tridiagonal form, solve the tridiagonal problem and back-transform the eigenvectors [26], [27]. On GPUs the reduction limits the eigensolve, because it needs a product of the trailing matrix with a vector for every column [28] and is therefore bound by memory bandwidth. On a V100 the dense eigensolver takes 12.4 times as long as a matrix product of the same order [29], and on an H100 the cuSOLVER tridiagonalization runs at 2.1 of 67 TFLOP/s [30]. In distributed memory, every column also needs collectives [31], whose latency does not shrink as GPUs are added. Two-stage methods move the reduction into matrix–matrix products but add bulge chasing and a second back-transformation of the eigenvectors [32], [33]. The ELSI developers note that this second back-transformation is not naturally suited to GPU acceleration [34]. The largest systems now use GPUs from three vendors [35], so a solver must run efficiently on all three.

We present HEXA, a dense Hermitian eigensolver for GPU systems (Fig. 1). It reduces the matrix, of order n, in one stage to a band of small width B and solves the band eigenproblem directly by divide and conquer, so there is no bulge chasing and only one back-transformation. EigenExa's `eigen_sx` does the same on CPUs with a pentadiagonal band [36], [37]. The band width balances the two dominant costs. The reduction takes n/B dependent steps, each a pass over the trailing matrix and, in distributed memory, a round of collectives, while the divide-and-conquer work grows with B. Each merge applies the B rank-one updates of the band in sequence, and for large merges a fast multipole method (FMM) built from batched matrix products reduces the cost of the resulting B eigenvector products. The best B depends on the matrix size, the number of GPUs and the machine, and in distributed memory HEXA selects it at run time, together with the data distribution of the reduction.

Our contributions are:

- an analysis of the band width as the parameter that trades the memory traffic and dependent steps of the reduction against the work of the divide and conquer (Section 3.1);
- a wide-band reduction that factors the columns of each step by CholeskyQR2 with Householder reconstruction, in real and complex arithmetic (Section 3.2);
- a GPU banded divide and conquer whose eigenvector updates are applied by a Cauchy fast multipole method in batched matrix products, with interpolation coordinates anchored at the poles and cluster trees split at gaps of the spectrum (Sections 3.3–3.4);
- a distributed algorithm with two distributions of the reduction (a replicated panel with one collective per step, and a two-dimensional panel with less data per process), and a run-time selection of the band width and the distribution from micro-benchmarks (Sections 4–5);
- an evaluation on NVIDIA GH200, AMD MI250X and Intel Max 1550 systems with up to 64 nodes, against ELPA, cuSOLVERMp, DLA-Future, EVD4SC2025 [38] and the vendor eigensolvers, in single and double precision, real and complex (Section 7). In FP64 strong scaling, HEXA is 4.9–10.0 times faster than ELPA's one-stage solver on the GH200 system, 3.0–7.6 times on the MI250X system and 3.8–6.9 times on the Max 1550 system, and it solves a problem of order $n = 524288$ in 150.4 s on 256 GH200s.

2 Background and Related Work

Two-stage reduction: Successive band reduction [32], [39] reduces the matrix first to a band with matrix–matrix operations and then to tridiagonal form by bulge chasing. It underlies ELPA's two-stage solver [33], [40], the tile algorithms of Haidar et al. [41] and the GPU solvers of MAGMA [42], [43].

Recent GPU solvers: Wang et al. find that the tridiagonalization accounts for over 97% of the cuSOLVER eigensolver time on an H100, and accelerate it with double-blocked band reduction and GPU bulge chasing [30], [44]. Earlier, Zhang et al. targeted the band reduction on tensor cores [45]. Wang et al. then reorganize the back-transformation through the bulge-chasing stage, which yields a full FP64 eigensolver with eigenvectors on average 2.21× faster than cuSOLVER [38]. We compare with its implementation, denoted EVD4SC2025 below. On eight A100s, a pipelined two-stage solver is reported on average 5.74× faster than cuSOLVERMp, and cuSOLVERMp, MAGMA and ELPA reach about 1.5% of peak there [46].

Distributed GPU eigensolvers: ELPA offers one- and two-stage solvers with GPU support [40], [47], [48]. In cuSOLVERMp, `syevd` combines a distributed tridiagonalization, divide and conquer and back-transformation on NVIDIA GPUs [49]. DLA-Future is a task-based library with a GPU-enabled distributed eigensolver [50], and SLATE is a distributed, GPU-accelerated dense linear algebra library [51]. We compare with ELPA, cuSOLVERMp and DLA-Future in Section 7.

Divide and conquer: Cuppen's method [52], [53] with the stable eigenvector computation of Gu and Eisenstat [54] underlies the divide-and-conquer solvers of LAPACK [26] and, through Tisseur and Dongarra [55], of ScaLAPACK. Divide and conquer has been extended to band matrices [56], [57] and block-tridiagonal matrices [58], [59]. EigenExa's `eigen_sx` reduces a dense matrix to a narrow (pentadiagonal) band and solves the band matrix by divide and conquer, without a tridiagonal stage [36], [37]. It is the prior design closest to HEXA and was developed for CPU systems.

Gu and Eisenstat already noted that the fast multipole method [60] reduces the cost of all eigenpairs to $\mathcal{O}(n^2)$ [54]. Livne and Brandt solve all roots of a secular equation in $\mathcal{O}(n)$ operations [61], and SuperDC applies rank-structured techniques to divide and conquer [62]. Liao et al. accelerate the eigenvector updates of distributed tridiagonal divide

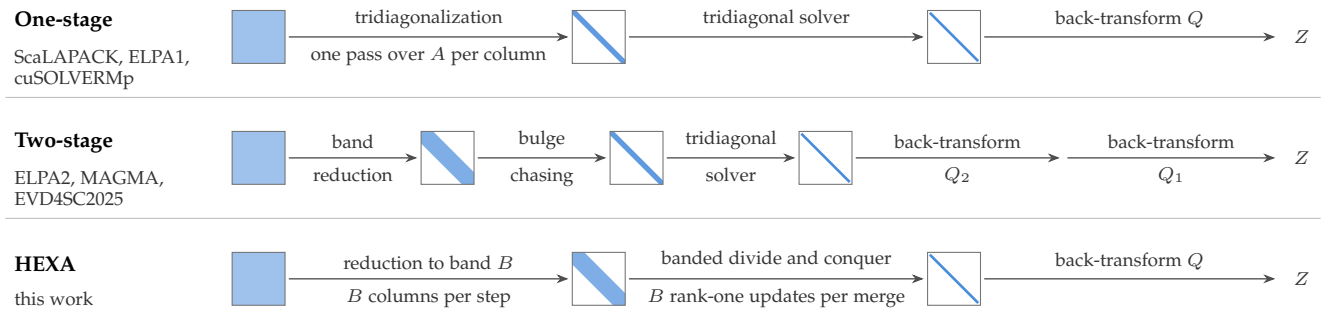


Fig. 1. Three routes to all eigenpairs of a dense Hermitian matrix. Tridiagonalization reads the trailing matrix once per column; two-stage solvers replace it with a matrix-product band reduction followed by bulge chasing, and apply both sets of reflectors to the eigenvectors. HEXA stops at a narrow band, solves it by divide and conquer, and applies one set of reflectors. ELPA1, ELPA2: ELPA's one- and two-stage solvers.

and conquer with structured (Cauchy-like) matrix multiplication [63], and Li et al. extend this to banded divide and conquer on CPU clusters [64]. We are not aware of a GPU implementation of banded divide and conquer or of such fast eigenvector updates.

Communication and panel factorization: Communication lower bounds for dense linear algebra [65] apply to the reduction. For tall-and-skinny matrices, CholeskyQR2 needs only matrix products and is accurate for condition numbers up to about $u^{-1/2}$, with u the unit roundoff [66], [67], and the Householder representation can be reconstructed from its orthonormal factor [68], [69].

3 Narrow-band reduction and banded divide and conquer

For a Hermitian matrix A of order n, HEXA computes the eigenvalues Λ and eigenvectors Z, $A = Z\Lambda Z^H$, in three stages. A one-stage Householder reduction turns A into a band matrix $C = Q^H AQ$ of semi-bandwidth B (the band width), with Q the product of the Householder reflectors; divide and conquer computes the eigendecomposition $C = Y\Lambda Y^H$; and the back-transformation forms $Z = QY$. In the distributed solver, P is the number of processes, one per GPU. This section analyzes the trade-off that B controls and then describes the three stages. The distributed algorithm follows in Section 4.

3.1 The band width trade-off

The reduction performs about $\frac{4}{3}n^3$ flops for any $B \ll n$, but on GPUs its cost is set by memory traffic and by dependent steps, and both decrease as B grows. Each step reduces B columns and needs one product of the trailing matrix with the B new reflectors, which reads the trailing matrix once. Summed over the n/B steps this is $\Theta(n^3/B)$ words, against $\Theta(n^3)$ for tridiagonalization, which needs one such pass per column. Each step also depends on the previous one through the factorization of its columns, and in a distributed solver it carries one or more collectives. A wider band makes the divide and conquer more expensive. A merge of two halves applies B rank-one updates (Section 3.3), and the eigenvector product of each update touches the whole eigenvector block of the merge. With the fast multipole method of Section 3.4, an update with K undeflated eigenvalues on an eigenvector block of r rows costs $\mathcal{O}(rK)$ instead of the $2rK^2$ flops of a matrix product. Level ℓ of the tree, with 2^ℓ merges of order $n/2^\ell$, therefore costs $\mathcal{O}(Bn^2/2^\ell)$, and the whole divide and conquer $\mathcal{O}(Bn^2)$. The back-transformation costs $2n^3$ flops in matrix products, independent of B.

The total time behaves as

$$T(B) \approx \frac{n}{B}\, s(n,P) + B\, d(n,P) + T_{\text{back}}(n,P), \qquad (1)$$

with s the cost of one reduction step and d the divide-and-conquer cost per unit of band width on P GPUs. The step cost s is bound by memory bandwidth on one GPU and by collective latency at scale, so it shrinks slowly, or not at all, as P grows, while d ideally divides over the GPUs. In this model the best band width grows with n and with the number of GPUs, and it depends on the machine. HEXA measures s and d at run time (Section 5).

3.2 Reduction to a narrow band

Narrow bands, $B \le 8$: The reduction follows the blocked tridiagonalization of LAPACK's `sytrd` [28], with B columns per step instead of one. EigenExa's `eigen_sx` reduces to a pentadiagonal matrix in the same way [36], [37]. The reduction proceeds in panels of 64 columns, and each step reduces B of them. A step factors the B columns below the band into Householder reflectors, kept as a block reflector $I - VTV^H$ with T upper triangular [70]. With A_{22} the trailing matrix, of order m, the step forms $S = A_{22}V$ and $G = V^H A_{22} V$, corrected for the pending updates of the panel, and accumulates $W = ST - \frac{1}{2}VT^H GT$. Once per panel, the trailing matrix receives the Hermitian update $A_{22} - VW^H - WV^H$ of all steps in the panel.

A single kernel computes the bandwidth-bound product S together with G. The kernel reads each tile of the lower triangle once, uses it for both its own and its transposed contribution to all B columns, and accumulates the results with atomic additions. Its tile size and thread-block size are tuned per GPU model (Section 6.1).

Wide bands, $B = 16, 32, 64$: For wide bands the product $S = A_{22}V$ is a matrix product with the full (Hermitian) trailing matrix, and the trailing updates are applied once per panel, here of 128 columns, as in the first stage of successive band reduction [32]. The factorization of the $m \times B$ block of columns in each step remains latency-bound, because Householder QR performs it as a sequence of B

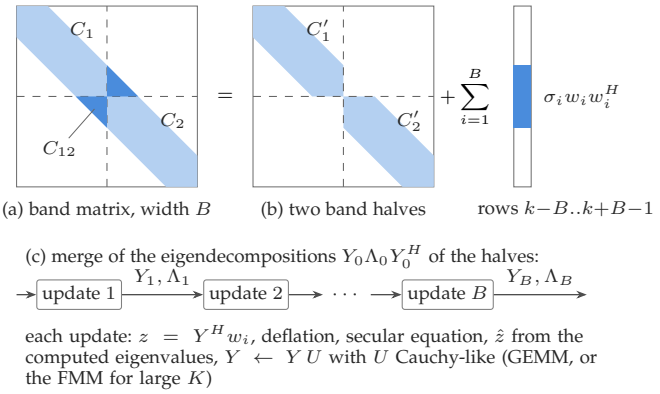


Fig. 2. Banded divide and conquer. (a)–(b) Splitting a band matrix of width B at row k leaves two band halves C'_1, C'_2 (the diagonal blocks of (2)) and B rank-one terms. The terms come from the singular value decomposition of the coupling block C_{12} and are supported on the $2B$ rows around the split. (c) A merge applies the rank-one terms one at a time.

matrix–vector operations. HEXA instead factors the block by CholeskyQR2 [66], [67] and recovers the Householder representation by the reconstruction of Ballard et al. [69] (LAPACK's `orhr_col` and `unhr_col`), in real and complex arithmetic. The factorization then costs a few matrix products (the Gram matrices) and $B \times B$ operations, and yields the `geqrf` representation that the back-transformation expects. If the Cholesky factorization breaks down, or the second Gram matrix deviates from the identity by more than 0.1 (columns too ill-conditioned for CholeskyQR2), the columns are refactored by `geqrf`.

A step also needs $B \times B$ inner products summed over all m rows, between the new reflectors V and each of S, the earlier reflectors of the panel and their W. HEXA computes them either by one GEMM or by a split-K scheme (a strided-batched GEMM over row blocks followed by a fixed-order sum), whichever the build-time autotuner finds faster on the device (Section 6.1).

3.3 Banded divide and conquer

Tearing: When the band matrix C is split at row k into diagonal blocks C_1 and C_2, the halves are coupled only by the $B \times B$ upper triangular block $C_{12} = C(k{:}k{+}B{-}1,\ k{-}B{:}k{-}1)$. With its singular value decomposition $C_{12} = \sum_{i=1}^{B} \sigma_i p_i q_i^H$ and $w_i = [q_i; p_i]$ on rows $k-B, \ldots, k+B-1$,

$$C = \operatorname{diag}\Big(C_1 - \sum_i \sigma_i q_i q_i^H,\ C_2 - \sum_i \sigma_i p_i p_i^H\Big) + \sum_i \sigma_i w_i w_i^H, \tag{2}$$

where the subtracted terms stay inside the band, so the halves are again band matrices (Fig. 2). Leaves of at most 256 rows are solved by the batched dense eigensolver of the vendor library.

Merges as sequences of rank-one updates: A merge applies the B terms of (2) as a sequence of rank-one updates, as in the block-tridiagonal divide and conquer of Gansterer et al. [59] and the rank-structured solver of Vogel et al., which covers banded matrices [57]. Gansterer et al. avoid solving the rank-B problem at once, citing Arbenz's observation that it loses the numerical orthogonality of the eigenvectors [56], [59]. In the basis of the current eigenvectors Y, which start as those of the two halves, each term is a Hermitian rank-one update $D+\rho zz^H$ of the diagonal matrix D of the current eigenvalues, with $\rho = \sigma_i$ and $z = Y^H w_i$. It is solved by the method of Gu and Eisenstat [54] as in LAPACK's `laed` routines [26]. After deflation, K eigenvalues remain, and they are the roots of a secular equation whose poles are the diagonal entries d_i of D. The update vector is then recomputed, as $\hat{z}$, from the computed eigenvalues, so that the eigenvectors are numerically orthogonal [54], [55]. The eigenvectors are updated as $Y \leftarrow YU$, with U the Cauchy-like eigenvector matrix of the update. This costs B eigenvector products per merge instead of one (Section 3.4).

On the GPU, the secular equations are solved by a port of LAPACK's `laed4`, `laed5` and `laed6` with their iteration and stopping rules unchanged. One 32-thread warp per eigenvalue forms the sums over the K poles in parallel. As in `laed4`, each eigenvalue is represented by one of its two neighboring poles and an offset from it, and every difference $d_i - \lambda_j$ is formed from these offsets.

Complex Hermitian matrices: For complex A the update vectors $z = Y^H w_i$ are complex. Since $D + \rho zz^H = \Phi\,(D + \rho|z|\,|z|^T)\,\Phi^H$ with $\Phi = \operatorname{diag}(z_j/|z_j|)$, scaling the columns of Y by Φ turns every update into a real one, so that deflation, the secular equation and U are real. The product of a complex Y with a real U is a real product with twice the rows, so a complex merge costs twice a real one rather than four times.

3.4 Fast multipole application of the eigenvector update

The entries $U_{ij} = \hat{z}_i s_j/(d_i - \lambda_j)$, with λ_j the updated eigenvalues and s_j normalizing column j, form a Cauchy-like matrix, whose blocks between well-separated sets of poles d_i and eigenvalues λ_j have low numerical rank [54]. HEXA applies U with a kernel-independent FMM based on Chebyshev interpolation [71] with nested bases. Its admissibility parameter is $\eta = 0.7$, with 24 interpolation points per box in double precision and 12 in single. Every FMM operation (leaf moments, translations between levels, far-field transfers, evaluation and the near field) is a batched GEMM whose long dimension is the r rows of Y. The setup of the FMM depends only on K and is shared by all rows.

Interpolation uses box-local coordinates whose origin is a pole, and each eigenvalue is kept as an offset from its nearest pole. The relative accuracy of eigenvalues close to a pole, which the secular solver preserves, is therefore not lost in the far field. A box is split at its median, or at its largest gap among the middle 40% of its points when that gap exceeds a quarter of the box width. Boxes then do not straddle the gaps of clustered spectra, and admissible pairs stay well separated. Non-admissible leaf pairs (at most 128 points per leaf) are evaluated exactly, with the same entries as the dense U. An update uses the FMM when it has at least K_0 undeflated eigenvalues, with K_0 measured per GPU model as the crossover with the dense product (Section 6.1), and smaller updates use one GEMM.

3.5 Back-transformation

The back-transformation $Z = QY$ applies the reflectors of the reduction in blocks with the compact WY representa-

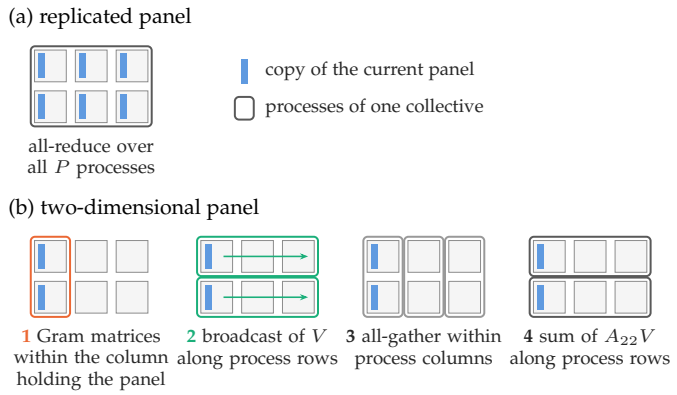


Fig. 3. The collectives of one reduction step on a 2×3 process grid. (a) With the replicated panel, a step performs one all-reduce of about mB elements over all P processes. (b) With the two-dimensional panel, it performs collectives within the process column that holds the panel (1), along process rows (2, 4) and within every process column (3), and moves about $(m/p_r + m/p_c)B$ elements per process.

tion [70], forming the triangular factor from the Gram matrix of the reflectors in the block [72], [73], so each block costs three matrix products. The block size is tuned per GPU model.

4 Distributed-memory algorithm

The distributed solver runs one MPI process per GPU on a $p_r \times p_c$ grid, $P = p_r p_c$, with A and Z in the two-dimensional block-cyclic distribution of ScaLAPACK [27] (block size n_b). Every matrix product in the algorithm has one thin dimension (the band width, a panel width or a back-transformation block) or is local to the rows owned by a process. Each stage therefore consists of local vendor BLAS calls and collectives on thin operands.

4.1 Reduction

In the standard two-dimensional tridiagonalization of ScaLAPACK and ELPA's one-stage solver [31], [33], every column requires several broadcasts and reductions along process rows and columns, so the reduction performs $\Theta(n)$ dependent rounds of collectives. A band reduction needs only n/B steps. HEXA offers two distributions of the work within a step, which trade communication volume against the number of collectives, as Fig. 3 shows.

Replicated panel: The columns of each panel are gathered to every process once. Every process then holds a copy of the current panel and of the thin matrices (V, W, the pending updates) and factors the columns of each step redundantly. Only the products with the trailing matrix are distributed. Each process computes the contribution of its local tiles to $S = A_{22}V$ (and, for $B \leq 8$, $G = V^H A_{22} V$), and one all-reduce over all P processes per step sums these contributions together with the inner products of the step. Because every sum of replicated data is formed in a fixed order, all processes compute bitwise identical reflectors and take the same branch decisions. The trailing update is local. A step thus costs one collective of about mB elements, and the reduction moves $\Theta(n^2)$ elements per process in total, independent of P.

Two-dimensional panel ($B = 16, 32$): This distribution does not replicate the panel. In the process column that holds the panel, the processes factor the columns of the step by CholeskyQR2, summing the Gram matrices over that process column and computing the Householder reconstruction redundantly. The reflectors are then broadcast along the process rows and redistributed to the column distribution by an all-gather within each process column. Finally, the product $S = A_{22}V$ is formed locally and summed along the process rows. In one variant the first Gram matrix of CholeskyQR2 is folded into the collective sum that already carries the inner products of the previous step, at the cost of extra arithmetic; in the other it is summed separately. A step moves about $(m/p_r + m/p_c)B$ elements per process, so the reduction moves $\Theta(n^2/\sqrt{P})$ elements per process on a square grid. Each step, however, performs five or six collectives instead of one, and two or three of them are within that process column. Which distribution is faster depends on n, P and the latency and bandwidth of the collectives on the machine, so HEXA selects it at run time together with B (Section 5). $B = 64$ uses the replicated panel.

4.2 Divide and conquer

Each process owns one subtree of the divide-and-conquer tree and solves it locally without communication. Above the subtrees, the rows of each node are split between its two children in proportion to the number of processes each child receives. Merges above that level are distributed by row blocks of Y. The B update vectors are projected onto Y once per merge (one all-reduce) and carried through all of its rank-one updates. Deflation is computed redundantly, the secular equations are divided among the processes and their results all-gathered, and the product YU is local to the rows owned by each process. A final all-to-all exchange moves Y from the row distribution to the block-cyclic one.

4.3 Back-transformation

The back-transformation overwrites the block-cyclic Y with $Z = QY$, one block column of n_b reflectors at a time. The reflectors are broadcast along the process rows from the process column that holds them, the products $V^H V$ and $V^H Z$ are formed locally and summed within each process column, and the update of Z is local. It moves $\Theta(n^2/\sqrt{P})$ elements per process in total.

5 Selecting the band width and distribution at run time

The model (1) explains the trade-off but cannot predict its optimum. The step cost s depends on how the trailing-matrix product and the collectives perform on the machine and the process grid, and the divide-and-conquer cost d on deflation and on the FMM. HEXA therefore calibrates the model at run time. The candidates are the band widths $B \in \{4, 8, 16, 32, 64\}$ and, for $B = 16$ and 32, the replicated panel and both variants of the two-dimensional panel (Section 4.1).

Step cost: Before its first solve of a given size, HEXA times one panel of the reduction for each candidate on the actual matrix and process grid, keeping the fastest of three runs. The per-step cost varies with the trailing size m through terms in 1, m and m^2. Three such micro-benchmarks, at the Gauss–Legendre points $m \in \{\frac{1}{2} - \frac{1}{2}\sqrt{0.6},\ \frac{1}{2},\ \frac{1}{2} + \frac{1}{2}\sqrt{0.6}\}\, n$, weighted $\frac{5}{18}, \frac{8}{18}, \frac{5}{18}$, therefore integrate it exactly over the reduction and give the estimated reduction time $\tilde{R}_c$ of candidate c.

Prediction: The first solve runs $B = 16$ with the replicated panel and measures its reduction time R_{16} and its divide-and-conquer time M_{16}. The predicted time of candidate c, with band width B_c, is

$$T_c = R_{16} - \tilde{R}_{16} + \tilde{R}_c + M_{16}\,\frac{B_c}{16}, \tag{3}$$

where the first two terms keep the measured cost of the candidate-independent parts of the reduction and the last scales the divide-and-conquer time linearly in B, as the analysis of Section 3.1 suggests. All later solves use the candidate with the smallest T_c.

6 Implementation

HEXA is written in C++17 for CUDA, HIP and SYCL from a single source. Each device kernel has one body, written against a small abstraction of the execution model (thread and block indices, 32-thread warps with shuffle operations, block barriers, shared memory, atomics). Dense operations use the vendor libraries (cuBLAS and cuSOLVER, rocBLAS and rocSOLVER, oneMKL), and the parameters that depend on the GPU model are found by the build-time autotuner described below.

All stages are generic in the scalar type and support single and double precision, real and complex. Collectives on device data go through one interface, implemented by the vendor collective library (NCCL, RCCL or oneCCL), by a GPU-aware MPI, or by MPI with buffers staged through host memory.

6.1 Build-time autotuning

Parameters that depend only on the GPU model are tuned once, when the library is built, by empirical search [74] over micro-benchmarks on synthetic data:

- the tile size, thread-block size and tile grouping of the narrow-band product kernel;
- the block size of the back-transformation, per matrix size;
- the FMM threshold K_0, as the crossover between FMM and GEMM rank-one updates;
- one GEMM or split-K for the inner products of the reduction.

The search takes 13.6 s on the GH200, 21.1 s on the MI250X and 29.5 s on the Max 1550 for all four precisions.

On the MI250X and the Max 1550 (Table 1), one vendor GEMM is up to 55.3 and 4.7 times slower than the split-K form. On the GH200 it takes 0.9–1.4 times as long as the split-K form and is the faster of the two in single precision.

TABLE 1
The inner-product micro-benchmark of the autotuner on one GH200, one MI250X GCD (graphics compute die) and one Max 1550 tile, for Gram matrices of 16 and 32 columns over 4096, 16384 and 65536 rows, summed time (ms) of one vendor GEMM / of the split-K form. Split-K is chosen when it takes less than 97% of the GEMM time; * one GEMM chosen; s, d: real single, double; c, z: complex single, double.

prec	GH200	MI250X GCD	Max 1550 tile
s	0.111 / 0.126*	4.739 / 0.239	0.962 / 0.321
d	0.090 / 0.083	11.579 / 0.358	1.049 / 0.355
c	0.176 / 0.127	26.178 / 0.473	0.635 / 0.360
z	0.156 / 0.129	0.572 / 0.413	2.046 / 0.437

7 Results

7.1 Systems and software

We evaluate HEXA on Hangang (KISTI-6) [75], Frontier [76] and Aurora [77], with GPUs from three vendors (Table 2). A node of Hangang has four NVIDIA GH200 superchips, each a 72-core Arm CPU coupled to a Hopper GPU [78]. A node of Frontier has one 64-core AMD EPYC CPU and four AMD MI250X, each with two graphics compute dies (GCDs) [79]. A node of Aurora has two Intel Xeon CPU Max processors and six Intel Data Center GPU Max 1550, each with two tiles [80], [81]. All three connect their nodes with HPE Slingshot. Every run uses one MPI process per GPU device that the software exposes (a GH200, a GCD or a tile), so a node holds 4, 8 or 12 processes.

HEXA uses NCCL on Hangang and RCCL on Frontier. On Aurora, because oneCCL was not stable for either solver, both HEXA and ELPA communicate through MPI with host-staged buffers (ELPA with eight OpenMP threads per process).

7.2 Matrices, baselines and methodology

As in LAPACK, s and d denote real single (FP32) and double (FP64) precision, and c and z complex single and double. All distributed runs use the same test matrix, a real symmetric matrix with entries in $[-1, 1)$ given by a hash of the global index pair, or in complex precision its Hermitian counterpart with a hashed imaginary part. The ELPA benchmark generates the identical matrix, so the eigenvalues can be compared directly. On one GPU, the *uniform* matrix has independent uniform entries in $(-1, 1)$. The other matrices are $X\Lambda X^H$, with X the orthonormal QR factor of a random matrix. The *clustered* matrix repeats each eigenvalue eight times, with the clusters spread uniformly over $[-1, 1]$. *Mo* ($n = 8400$) prescribes the 8400 Kohn–Sham eigenvalues, computed with DFT-FE [14], of the $8 \times 8 \times 8$ body-centered cubic molybdenum supercell with a vacancy (1023 atoms) of the DFT-FE benchmark set [82].

ELPA [47] runs its one-stage solver everywhere. Its two-stage solver, also run on Hangang and Aurora, is slower than its one-stage solver at all but one strong-scaling point. On Frontier it was 3.3–4.8× slower on one node and was not run at scale. On Hangang, cuSOLVERMp [49] and DLA-Future [50] run as well. EVD4SC2025, the two-stage GPU solver of Wang et al. [38], runs on one GH200. On one GPU we compare with the dense eigensolver of each vendor library (`syevd/heevd`). The block size n_b is 128 on Hangang

TABLE 2
Systems and software. Device figures are per MPI process (one GH200, one MI250X GCD, one Max 1550 tile); the FP64 peak of the Max 1550 is derived from its 32,768 FP64 operations per clock at 1.6 GHz [77].

	Hangang (KISTI-6)	Frontier (OLCF)	Aurora (ALCF)
node	4 GH200 (72-core Arm Neoverse V2 CPU and Hopper GPU each)	1 AMD EPYC (64 cores), 4 MI250X (8 GCDs)	2 Intel Xeon CPU Max, 6 Max 1550 (12 tiles)
network per node	4 Slingshot NICs	4 Slingshot NICs, 200 Gb/s each	8 Slingshot 11 endpoints
device memory	96 GB HBM3, 4 TB/s	64 GB HBM2e, 1.6 TB/s	64 GB HBM2e (128 GB and 3.3 TB/s per GPU)
FP64 peak (TFLOP/s)	34 (67 with tensor cores)	23.9 (47.9 with matrix cores)	26.2 (52.4 per GPU)
compilers	CUDA 13.2 (NVIDIA HPC SDK 26.5), GCC 14.2	ROCm 7.2.0, GCC 14.2	oneAPI DPC++ 2026.1, GCC 14.3
MPI	Cray MPICH 9.1.0	Cray MPICH 9.1.0	MPICH 5.0.0
vendor libraries	cuBLAS, cuSOLVER (CUDA 13.2)	rocBLAS, rocSOLVER (ROCm 7.2.0)	oneMKL (oneAPI 2026.1)
HEXA collectives	NCCL 2.30.7	RCCL 2.27.7	MPI, host-staged
baselines	ELPA 2026.02.002 (one- and two-stage), cuSOLVERMp 0.8.0, DLA-Future 0.10.0, EVD4SC2025, cuSOLVER	ELPA 2026.02.002 (one-stage), rocSOLVER	ELPA 2026.02.002 (one- and two-stage), oneMKL

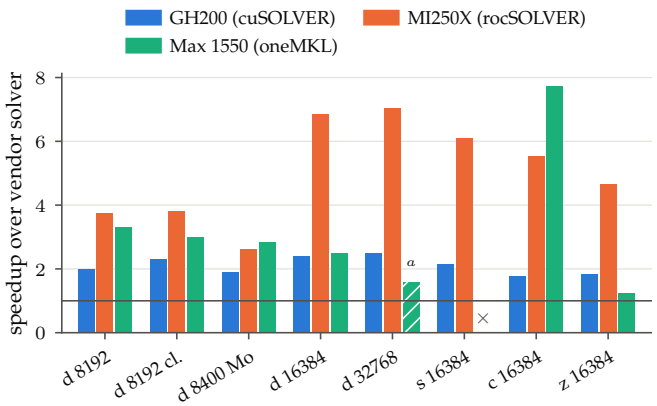


Fig. 4. Speedup of HEXA over the vendor eigensolver on one GPU ($B = 4$); the horizontal line marks equal time. Labels give the precision (d, s, c, z) and n; cl.: clustered spectrum, Mo: molybdenum spectrum, otherwise the uniform matrix (Section 7.2). [a] Hatched: the oneMKL result is wrong (residual 0.62); $\times$: oneMKL aborts.

and 64 elsewhere for ELPA, 512 for cuSOLVERMp, 1024 for DLA-Future, and 256 on Hangang and 128 elsewhere for HEXA. ELPA runs with its GPU kernels on all three systems. Each solver runs on its own near-square process grid, and the ELPA grid has no more rows than columns.

Each HEXA run is a first solve, which in distributed memory also selects the band width and distribution (Section 5), followed by up to three timed solves in the same program. The baselines run a warm-up and up to three timed solves. We report the mean of the timed solves and the stage times of process 0, separated by barriers. The speedup of HEXA over a solver is the time of that solver divided by the time of HEXA. Points marked † in the appendix tables were run with the preceding version of HEXA, which executes the same code for them. The appendix tabulates the results (Tables 3–9).

7.3 One GPU

On one GPU in FP64 (Fig. 4), HEXA is 1.9–2.5 times faster than cuSOLVER on the GH200, 2.6–7.0 times faster than rocSOLVER on an MI250X GCD and 2.5–3.3 times faster than oneMKL on a Max 1550 tile. In the other precisions it is 1.5–2.2 times faster on the GH200, 2.9–6.1 times on the MI250X GCD and 1.2–7.7 times on the Max 1550 tile.

Fig. 5 compares HEXA with EVD4SC2025 [38] stage by stage. Its divide and conquer, which runs on the host (hatched), is set aside here. At $n = 32768$, the tridiagonal reduction of EVD4SC2025 is faster than the one-stage reduction of HEXA to band width 4. For $n \geq 16384$, however, the two back-transformations of EVD4SC2025 alone take longer than the complete solve of HEXA. At $n = 32768$ they take 12.4 s, more than three times its reduction (3.6 s).

7.4 Strong scaling

Against ELPA's one-stage solver (Fig. 6), HEXA is 4.9–10.0 times faster on Hangang, 3.0–7.6 times on Frontier and 3.8–6.9 times on Aurora, at every strong-scaling point where ELPA ran. On Hangang, where cuSOLVERMp, DLA-Future and ELPA's two-stage solver also ran, HEXA is faster than each of them at every point (Fig. 7).

On Hangang and Frontier, strong scaling ends where the reduction stops shrinking (Fig. 8); on Aurora the solve keeps improving up to 32 nodes at every size. At $n = 65536$ the solve is fastest on 8 nodes on Hangang (4.77 s) and Frontier (8.39 s), and beyond 8 nodes the reduction takes longer. Its n/B dependent steps each carry collectives, and their cost grows with the node count. In profiles of process 0 on Hangang at $n = 131072$ (whole run, first solve included), the kernels of the collective library take 41% of the GPU kernel time on 8 nodes and 75% on 32; this share includes waiting for other processes.

On Frontier the divide-and-conquer time also grows at the largest size, from 14.0 s on 8 nodes to 42.6 s on 32 at $n = 262144$. At the same size and node counts it shrinks from 11.8 to 3.8 s on Hangang and from 18.8 to 9.1 s on Aurora. In the first solve, which runs $B = 16$ at both sizes, the merges on Frontier take about 2 s at $n = 131072$ on 8–32 nodes but 6.5, 17.5 and 42.3 s on 8, 16 and 32 nodes at $n = 262144$ (Section 8).

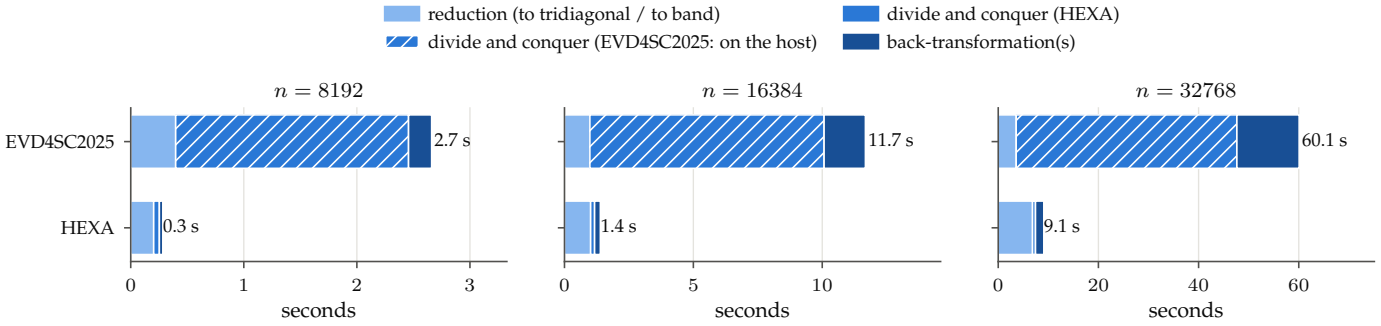


Fig. 5. The stages of EVD4SC2025 [38] and of HEXA on one GH200 in FP64. EVD4SC2025 runs its tridiagonal divide and conquer on the host (hatched).

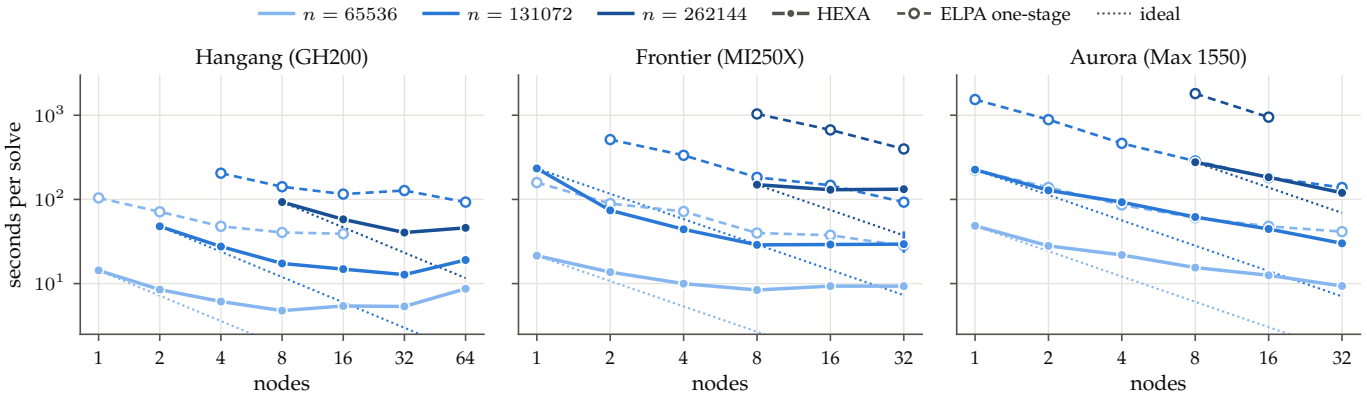


Fig. 6. Time per solve in FP64 strong scaling of HEXA (solid) and of ELPA's one-stage solver (dashed); dotted lines show ideal scaling from the first point of HEXA, and vertical bars give the range over the timed solves of HEXA.

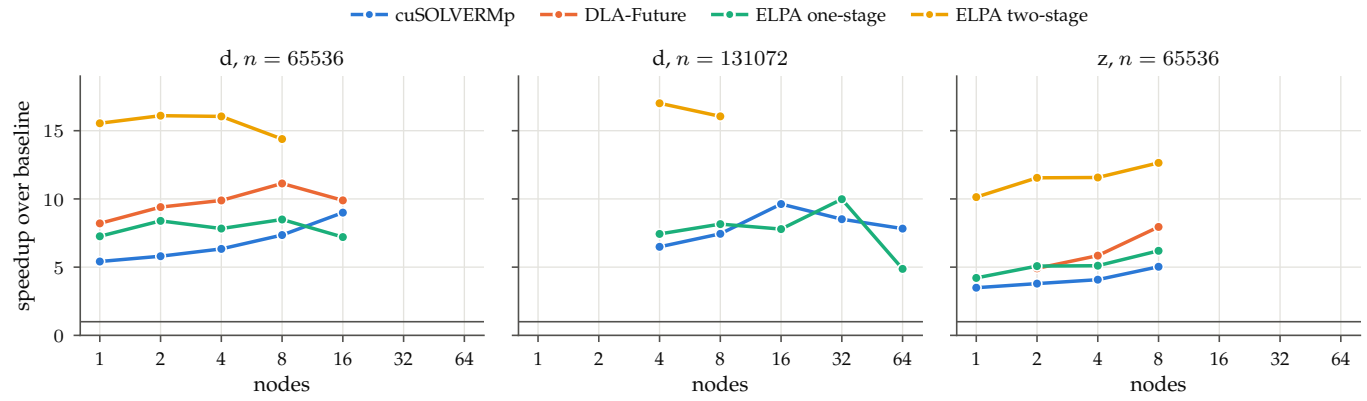


Fig. 7. Speedup of HEXA over each baseline on Hangang at every point where the baseline ran. DLA-Future did not complete at $n = 131072$; ELPA's two-stage solver ran up to 8 nodes.

7.5 Weak scaling and other precisions

With the matrix per GPU held constant, $n = 65536\sqrt{N}$ on N nodes, the work per GPU grows as $\sqrt{N}$. We therefore report the weak-scaling efficiency $T_1\sqrt{N}/T$, with T the time per solve and T_1 the time on one node (Fig. 9). The efficiency stays at or above 94% on Hangang and 100% on Aurora up to 32 nodes and $n = 370688$. On Frontier it stays at or above 96% up to 8 nodes and drops to 50% on 32. Relative to the 8-node run, the loss comes mostly from the divide and conquer on 16 nodes, and equally from the divide and conquer and the reduction on 32.

In the other precisions (Fig. 10) HEXA is 3.0–9.5 times faster than ELPA's one-stage solver in FP32 on Frontier and 4.4–5.0 times on Aurora, and in complex single 2.3–4.8 times faster on Frontier and 3.0–4.3 times on Aurora. In complex double it is 4.2–6.2 times faster on Hangang, 1.9–5.8 times on Frontier and 2.7–3.5 times on Aurora.

The timed solves of HEXA vary more than those of the baselines at some points (Figs. 6 and 10). The coefficient of variation stays below 2% for every baseline but reaches 43% for HEXA on Frontier, in complex double at $n = 65536$ on 32 nodes (12.9–29.8 s).

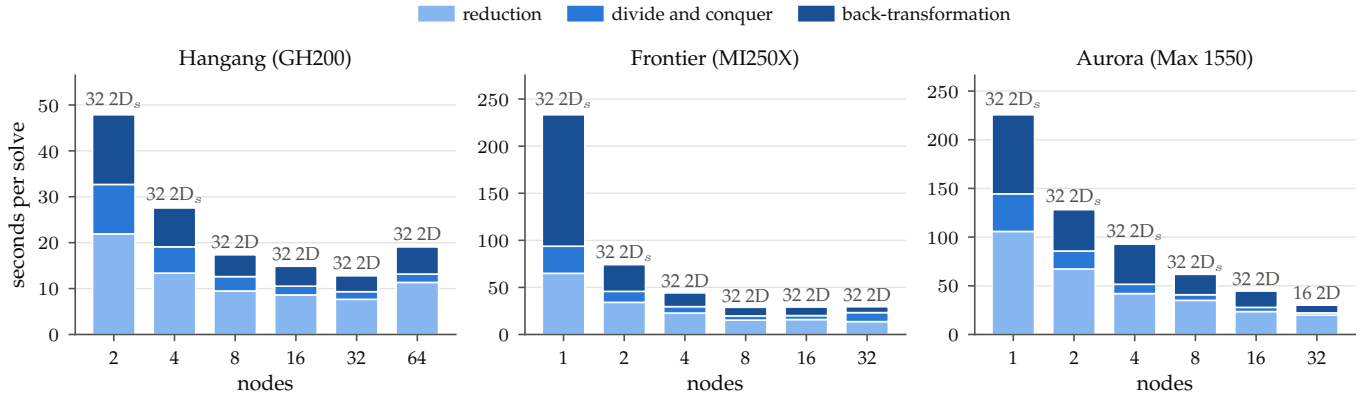


Fig. 8. Stage times of process 0 at $n = 131072$, FP64. Bar labels give the band width and reduction distribution selected at run time (2D: two-dimensional panel with the folded Gram matrix; 2D$_s$: with the Gram matrix summed separately).

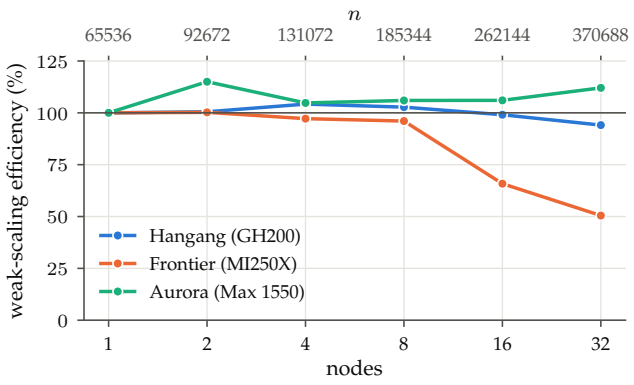


Fig. 9. FP64 weak-scaling efficiency $T_1\sqrt{N}/T$ with $n = 65536\sqrt{N}$ on N nodes, where T is the time per solve and T_1 the time on one node (the work per GPU grows as $\sqrt{N}$).

7.6 The run-time selection

The selection picks $B = 32$ at nearly every multi-node point on Hangang and Frontier, and $B = 16$ or 32 on Aurora, mostly with the two-dimensional panel (Fig. 8). Equation (3) predicts the reduction and divide-and-conquer time of the selected candidate to within a factor of 1.25 in 82% of the 204 runs with more than one process (Fig. 11a); the median ratio of measured to predicted time is 0.99 on Hangang, 1.02 on Frontier and 0.93 on Aurora. The largest deviations are on Aurora, where the slow replicated panel of the first solve makes the term $R_{16} - \tilde{R}_{16}$ of (3) as large as the whole reduction of the selected candidate.

The first solve costs 1.0–1.8 times a later solve on Hangang, 1.1–1.4 on Frontier and 1.6–5.8 on Aurora (Fig. 11b). On Aurora the per-step all-reduce of the replicated panel through host memory is expensive. At $n = 370688$ on 32 nodes, the reduction of the first solve takes 1237 of its 1344 s.

7.7 Accuracy

Every distributed HEXA run checks its result. Up to $n = 16384$ it computes the full residual $\|AZ - Z\Lambda\|/\|A\|$ and the orthogonality $\max|Z^H Z - I|$ and compares the eigenvalues with those of the vendor solver on process 0. Above that size, it checks 64 sampled eigenpairs and compares the sum of the eigenvalues with the trace. The eigenvalues are also compared with those saved by ELPA. All 222 runs of the distributed solver passed these checks. In double and complex double precision, the residuals $\|AZ - Z\Lambda\|/\|A\|$ and orthogonality errors stay below 10^{-13} on one GPU and 10^{-12} in the distributed runs. In single and complex single precision they stay below 10^{-4}. In all four precisions the eigenvalues match those of the vendor eigensolver on one GPU, for the uniform, clustered and Mo matrices, and those of ELPA in the distributed runs. They agree with ELPA as closely as cuSOLVERMp, DLA-Future and ELPA itself on other process counts do.

8 Discussion and Limitations

Strong scaling: Neither a wider band nor the two-dimensional panel removes the cost of the collectives that each step carries, so on Hangang and Frontier strong scaling ends at 8–32 nodes for $n = 65536$–131072.

Divide and conquer at scale: We have not yet identified the cause of the divide-and-conquer growth on Frontier at $n = 262144$, which does not occur on Hangang up to 64 nodes or on Aurora up to 32.

The first solve: The selection of band width and distribution costs one slower solve and the micro-benchmarks before it (Fig. 11b). This cost is incurred once per matrix size, so it matters for a single solve but is amortized over the repeated solves of an iterative application such as an SCF loop. When a good band width and distribution are known in advance, they can be set directly and the measurement skipped.

Scope of the evaluation: The distributed runs use one family of random matrices, and spectra with heavy deflation or tight clusters were tested on one GPU only. We report how well the time of the selected configuration was predicted but not how far it is from the fastest one, and we do not isolate the contributions of the fast multipole method, CholeskyQR2 and the two-dimensional panel.

9 Conclusion

We stop the reduction at a narrow band and solve the band problem by divide and conquer, which removes bulge chas-

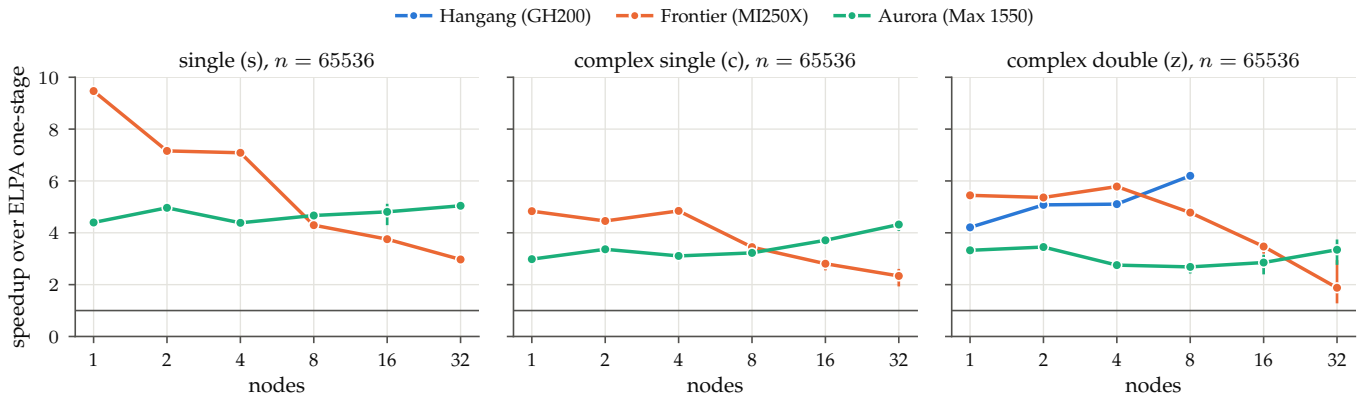


Fig. 10. Speedup of HEXA over ELPA's one-stage solver at $n = 65536$ in single precision and in complex single and double precision; vertical bars span the speedups of the slowest and fastest timed solves of HEXA. On Hangang ELPA ran only in complex double at this size.

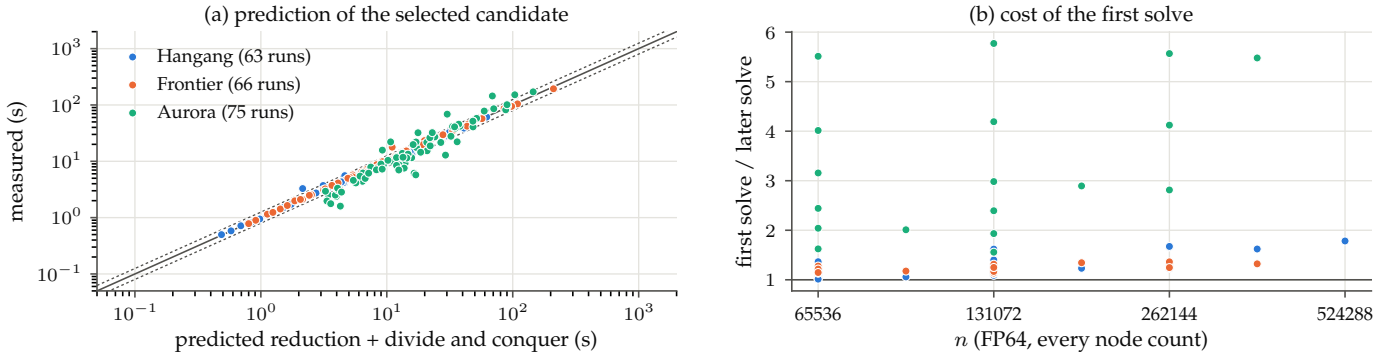


Fig. 11. (a) Predicted reduction and divide-and-conquer time of the selected candidate, from (3), against the measured mean of the later solves, for the runs with more than one process; dotted lines mark a factor of 1.25 either way. (b) The first solve, without the micro-benchmarks before it, against the mean of the later solves, FP64; colors as in (a).

ing and the second back-transformation from a dense GPU eigensolver. The band width then balances the dependent steps of the reduction against the work of the divide and conquer. HEXA chooses the band width and the distribution of the reduction from run-time measurements, so the same code adapts to three GPU architectures and to 1 to 64 nodes. On GH200, MI250X and Max 1550 systems, HEXA is faster than ELPA, cuSOLVERMp, DLA-Future, EVD4SC2025 and the vendor eigensolvers at every point and in every precision compared, with eigenvalues that agree with theirs.

Availability: HEXA and the benchmark harness are open source and available at https://github.com/knikhil1995/hexa and https://github.com/knikhil1995/hexa-benchmarks.

Appendix A
Detailed results

The tables below give the numbers shown in Figs. 4–10 and selected points of Fig. 11b. They are generated, like the figures, from the benchmark logs (Section 7.2).

TABLE 3
Time per solve of HEXA in seconds on one GPU (one GCD, one tile; $B = 4$) and the speedup of HEXA over the vendor eigensolver ($\times$). Matrices (Section 7.2): uniform random entries; cl.: eigenvalues in clusters of eight; Mo: a prescribed molybdenum spectrum. [a] The oneMKL result is wrong (residual 0.62). [b] Not measured: oneMKL aborts.

		GH200		MI250X GCD		Max 1550 tile	
prec	n	HEXA	cuSOLVER	HEXA	rocSOLVER	HEXA	oneMKL
d	8192	0.28	2.0×	0.75	3.8×	1.30	3.3×
d	8192 cl.	0.27	2.3×	0.72	3.8×	1.30	3.0×
d	8400 Mo	0.32	1.9×	0.80	2.6×	1.43	2.8×
d	16384	1.39	2.4×	3.42	6.8×	7.07	2.5×
d	32768	9.10	2.5×	21.81	7.0×	48.66	1.6×[a]
s	8192	0.22	2.2×	0.52	2.9×	1.10	6.3×
s	16384	1.01	2.2×	2.19	6.1×	–	–[b]
c	8192	0.45	1.5×	0.91	3.2×	2.17	6.1×
c	16384	2.25	1.8×	4.55	5.5×	11.28	7.7×
z	8192	0.69	1.6×	1.58	4.0×	3.70	1.4×
z	16384	3.79	1.8×	8.96	4.7×	20.32	1.2×

TABLE 4
The stages of EVD4SC2025 [38] (reduction to tridiagonal by band reduction and bulge chasing; the two back-transformations; its tridiagonal divide and conquer, which runs on the host) and of HEXA ($B = 4$) on one GH200 in FP64, in seconds (the faster of two runs).

	EVD4SC2025			HEXA			
n	to tridiag.	back	D&C (host)	to band	D&C	back	total
8192	0.40	0.20	2.06	0.20	0.05	0.03	0.28
16384	0.98	1.60	9.09	1.02	0.15	0.22	1.39
32768	3.59	12.43	44.06	6.85	0.59	1.66	9.10

TABLE 5
Time per solve of HEXA in FP64 strong scaling (s; mean of the timed solves), the band width and reduction distribution selected by the first solve (R: replicated panel; 2D: two-dimensional panel with the folded Gram matrix; $2D_s$: with the Gram matrix summed separately), and the time of ELPA's one-stage solver (ELPA1) and the speedup of HEXA over it; red., D&C, back: the three stages on process 0; –: not run or failed. † Run with the preceding version of HEXA (Section 7.2).

n	nodes	HEXA	red.	D&C	back	B		ELPA1	speedup
Hangang (GH200)									
65536	1	14.36	7.91	2.57	3.88	16	R	104.2	7.3×
	2	8.49	3.63	2.78	2.08	32	$2D_s$	71.26	8.4×
	4	6.12	3.04	1.60	1.48	32	R	47.94	7.8×
	8	4.77	2.70	1.01	1.06	32	$2D_s$	40.52	8.5×
	16	5.43	3.14	0.84	1.45	32	2D	39.08	7.2×
	32	5.35	2.81	0.91	1.64	32	2D	–	–
	64	8.69	5.05	1.04	2.61	32	R	–	–
131072	2	47.86	21.93	10.76	15.17	32	$2D_s$	–	–
	4	27.55	13.37	5.73	8.45	32	$2D_s$	204.8	7.4×
	8	17.36	9.48	3.13	4.75	32	2D	141.5	8.2×
	16	14.85	8.62	1.90	4.33	32	2D	115.7	7.8×
	32	12.78	7.66	1.66	3.45	32	2D	127.5	10.0×
	64	19.07	11.35	1.82	5.90	32	2D	92.91	4.9×
262144	8	93.11	49.28	11.82	32.01	32	$2D_s$	–	–
	16	57.96	32.66	6.20	19.10	32	2D	–	–
	32	40.42	24.47	3.78	12.16	32	$2D_s$	–	–
	64	45.98	28.95	3.46	13.57	32	2D	–	–
524288	64	150.4	92.81	7.75	49.84	32	2D	–	–
Frontier (MI250X)									
65536	1	21.45†	9.49	5.78	6.18	32	$2D_s$	158.5	7.4×
	2	13.69†	6.42	3.43	3.84	32	2D	89.34	6.5×
	4	9.99†	5.26	2.22	2.50	32	2D	71.85	7.2×
	8	8.39†	4.58	1.86	1.95	32	2D	39.92	4.8×
	16	9.34†	5.80	1.69	1.84	32	R	37.53	4.0×
	32	9.33†	5.16	2.49	1.68	32	2D	28.12	3.0×
131072	1	233.3†	64.94	28.77	139.6	32	$2D_s$	–	–
	2	74.14†	34.13	11.67	28.35	32	$2D_s$	514.0	6.9×
	4	44.14†	22.74	6.85	14.54	32	2D	334.2	7.6×
	8	28.86†	15.24	4.47	9.14	32	2D	182.9	6.3×
	16	29.11†	15.88	4.19	9.04	32	2D	147.3	5.1×
	32	29.44†	13.67	9.58	6.18	32	2D	92.73	3.2×
262144	8	149.6†	73.11	13.98	62.52	32	2D	1035.8	6.9×
	16	130.3†	58.11	36.38	35.82	32	2D	669.4	5.1×
	32	132.3†	62.00	42.57	27.71	16	2D	397.4	3.0×
Aurora (Max 1550)									
65536	1	48.58	25.69	6.56	16.33	16	$2D_s$	222.4	4.6×
	2	28.09	16.83	2.97	8.28	16	2D	139.0	4.9×
	4	21.88	11.73	1.65	8.50	16	2D	85.87	3.9×
	8	15.50	10.23	1.21	4.07	16	2D	60.65	3.9×
	16	12.55	8.13	1.03	3.39	16	2D	47.98	3.8×
	32	9.37	6.58	0.99	1.80	16	2D	41.41	4.4×
131072	1	225.6	105.8	38.67	81.12	32	$2D_s$	1536.6	6.8×
	2	128.2	67.23	18.46	42.46	32	$2D_s$	886.0	6.9×
	4	92.74	41.99	9.56	41.20	32	$2D_s$	463.4	5.0×
	8	61.73	35.02	5.79	20.91	32	$2D_s$	285.9	4.6×
	16	44.52	23.51	4.33	16.68	32	2D	181.9	4.1×
	32	30.06	19.97	2.21	7.89	16	2D	138.4	4.6×
262144	8	276.7	152.2	18.83	105.7	32	2D	1807.2	6.5×
	16	183.3	89.66	11.43	82.23	32	$2D_s$	948.6	5.2×
	32	119.8	68.90	9.08	41.78	32	2D	–	–

TABLE 6
Time per solve of HEXA on Hangang (GH200, 4 per node) in seconds and the speedup of HEXA over each baseline, at every point where the baseline ran; ELPA1, ELPA2: ELPA's one- and two-stage solvers. –: not run, out of memory, or failed (Section 7.2).

prec, n	nodes	HEXA	cuSOLVERMp	DLA-Future	ELPA1	ELPA2
d, 65536	1	14.36	5.4×	8.2×	7.3×	15.5×
	2	8.49	5.8×	9.4×	8.4×	16.1×
	4	6.12	6.3×	9.9×	7.8×	16.0×
	8	4.77	7.4×	11.1×	8.5×	14.4×
	16	5.43	9.0×	9.9×	7.2×	–
d, 131072	2	47.86	–	–	–	–
	4	27.55	6.5×	–	7.4×	17.0×
	8	17.36	7.4×	–	8.2×	16.1×
	16	14.85	9.6×	–	7.8×	–
	32	12.78	8.5×	–	10.0×	–
	64	19.07	7.8×	–	4.9×	–
z, 65536	1	44.00	3.5×	–	4.2×	10.1×
	2	23.35	3.8×	4.9×	5.1×	11.5×
	4	14.32	4.1×	5.8×	5.1×	11.6×
	8	9.00	5.0×	7.9×	6.2×	12.6×

TABLE 7
FP64 weak scaling of HEXA ($n = 65536\sqrt{N}$ rounded down to a multiple of 512 on N nodes, so n^2/P is constant), with its time per solve T in seconds, its band width and distribution (as in Table 5), and its weak-scaling efficiency $T_1\sqrt{N}/T$ in percent, with T_1 the time on one node (the work per GPU grows as $\sqrt{N}$). † Run with the preceding version of HEXA (Section 7.2).

system	nodes	n	HEXA	B	eff. (%)
Hangang	1	65536	14.36	16 R	100
	2	92672	20.20	32 $2D_s$	101
	4	131072	27.55	32 $2D_s$	104
	8	185344	39.52	32 $2D_s$	103
	16	262144	57.96	32 2D	99
	32	370688	86.32	32 2D	94
Frontier	1	65536	21.45†	32 $2D_s$	100
	2	92672	30.26†	32 2D	100
	4	131072	44.14†	32 2D	97
	8	185344	63.17†	32 2D	96
	16	262144	130.3†	32 2D	66
	32	370688	240.4†	16 2D	50
Aurora	1	65536	48.58	16 $2D_s$	100
	2	92672	59.75	16 $2D_s$	115
	4	131072	92.74	32 $2D_s$	105
	8	185344	129.7	32 $2D_s$	106
	16	262144	183.3	32 $2D_s$	106
	32	370688	245.3	32 2D	112

TABLE 8
Single precision (s), complex single (c) and complex double (z) at $n = 65536$, with the time per solve of HEXA in seconds and the speedup of HEXA over ELPA's one-stage solver (ELPA1); –: not run. † Run with the preceding version of HEXA (Section 7.2).

		s		c		z	
system	nodes	HEXA	ELPA1	HEXA	ELPA1	HEXA	ELPA1
Hangang	1	11.62†	–	33.28	–	44.00	4.2×
	2	6.92†	–	17.65	–	23.35	5.1×
	4	5.12†	–	11.08	–	14.32	5.1×
	8	4.06†	–	7.48	–	9.00	6.2×
	16	4.84†	–	6.77	–	8.59	–
	32	4.47†	–	6.05	–	7.49	–
	64	6.24†	–	8.81	–	10.40	–
Frontier	1	12.24†	9.5×	32.99	4.8×	49.69	5.4×
	2	8.72†	7.2×	20.63	4.5×	29.04	5.4×
	4	6.76†	7.1×	14.32	4.8×	18.96	5.8×
	8	6.27†	4.3×	11.18	3.4×	14.26	4.8×
	16	6.50†	3.8×	12.25	2.8×	15.72	3.5×
	32	7.49†	3.0×	11.88	2.3×	20.28	1.9×
Aurora	1	28.51	4.4×	62.21	3.0×	106.0	3.3×
	2	16.06	5.0×	33.67	3.4×	61.03	3.5×
	4	12.14	4.4×	24.29	3.1×	45.95	2.8×
	8	9.11	4.7×	16.80	3.2×	31.77	2.7×
	16	8.11	4.8×	12.27	3.7×	23.72	2.9×
	32	6.74	5.0×	9.35	4.3×	16.88	3.3×

TABLE 9
The first solve, which runs $B = 16$ with the replicated panel and selects the band width and distribution of the later ones, against the mean of the later solves, FP64 (s); red.: reduction in the first solve.

system	n	nodes	first	red.	timed	ratio
Hangang	65536	8	5.96	4.18	4.77	1.2
	65536	32	7.30	5.26	5.35	1.4
	131072	32	20.69	16.21	12.78	1.6
	262144	32	67.10	52.87	40.42	1.7
	370688	32	140.0	106.7	86.32	1.6
	524288	64	268.2	213.1	150.4	1.8
Frontier	65536	8	10.21	7.23	8.39	1.2
	65536	32	11.90	8.79	9.33	1.3
	131072	32	34.38	26.53	29.44	1.2
	262144	32	172.8	99.01	132.3	1.3
	370688	32	317.9	199.2	240.4	1.3
Aurora	65536	8	48.92	39.56	15.50	3.2
	65536	32	51.62	42.50	9.37	5.5
	131072	32	173.5	156.0	30.06	5.8
	262144	32	666.7	614.5	119.8	5.6
	370688	32	1343.9	1236.9	245.3	5.5

REFERENCES

[1] B. O'Donoghue, E. Chu, N. Parikh, and S. Boyd, "Conic optimization via operator splitting and homogeneous self-dual embedding," *Journal of Optimization Theory and Applications*, vol. 169, no. 3, pp. 1042–1068, 2016.

[2] M. Garstka, M. Cannon, and P. Goulart, "COSMO: A conic operator splitting method for convex conic problems," *Journal of Optimization Theory and Applications*, vol. 190, no. 3, pp. 779–810, 2021.

[3] Q-Chem, Inc., "Q-Chem 7.0 user's manual, section 4.6.1: Large molecules and linear scaling methods," https://manual.q-chem.com/latest/Ch4.S6.SS1.html, accessed 2026-11-12.

[4] C. Seidl and G. M. J. Barca, "Q-Next: A fast, parallel, and diagonalization-free alternative to direct inversion of the iterative subspace," *Journal of Chemical Theory and Computation*, vol. 18, no. 7, pp. 4164–4176, 2022.

[5] S. Jin, K. J. Roche, I. Stetcu, I. Abdurrahman, and A. Bulgac, "The LISE package: Solvers for static and time-dependent superfluid local density approximation equations in three dimensions," *Computer Physics Communications*, vol. 269, p. 108130, 2021.

[6] H.-J. M. Shi *et al.*, "A distributed data-parallel PyTorch implementation of the distributed Shampoo optimizer for training neural networks at-scale," 2023, arXiv:2309.06497.

[7] N. Vyas *et al.*, "SOAP: Improving and stabilizing Shampoo using Adam for language modeling," in *International Conference on Learning Representations (ICLR)*, 2025.

[8] M. Shao, F. H. da Jornada, C. Yang, J. Deslippe, and S. G. Louie, "Structure preserving parallel algorithms for solving the Bethe–Salpeter eigenvalue problem," *Linear Algebra and its Applications*, vol. 488, pp. 148–167, 2016.

[9] F. Pietracaprina, N. Macé, D. J. Luitz, and F. Alet, "Shift-invert diagonalization of large many-body localizing spin chains," *SciPost Physics*, vol. 5, no. 5, p. 045, 2018.

[10] Y. Zhou, Y. Saad, M. L. Tiago, and J. R. Chelikowsky, "Self-consistent-field calculations using Chebyshev-filtered subspace iteration," *Journal of Computational Physics*, vol. 219, no. 1, pp. 172–184, 2006.

[11] A. S. Banerjee, L. Lin, W. Hu, C. Yang, and J. E. Pask, "Chebyshev polynomial filtered subspace iteration in the discontinuous Galerkin method for large-scale electronic structure calculations," *The Journal of Chemical Physics*, vol. 145, no. 15, p. 154101, 2016.

[12] V. W.-z. Yu *et al.*, "ELSI: A unified software interface for Kohn–Sham electronic structure solvers," *Computer Physics Communications*, vol. 222, pp. 267–285, 2018.

[13] V. Gavini *et al.*, "Roadmap on electronic structure codes in the exascale era," *Modelling and Simulation in Materials Science and Engineering*, vol. 31, no. 6, p. 063301, 2023.

[14] S. Das, P. Motamarri, V. Subramanian, D. M. Rogers, and V. Gavini, "DFT-FE 1.0: A massively parallel hybrid CPU-GPU density functional theory code using finite-element discretization," *Computer Physics Communications*, vol. 280, p. 108473, 2022.

[15] W. Hu *et al.*, "High performance computing of DGDFT for tens of thousands of atoms using millions of cores on Sunway TaihuLight," *Science Bulletin*, vol. 66, no. 2, pp. 111–119, 2021.

[16] B. Hourahine *et al.*, "DFTB+, a software package for efficient approximate density functional theory based atomistic simulations," *The Journal of Chemical Physics*, vol. 152, no. 12, p. 124101, 2020.

[17] A. Levitt and M. Torrent, "Parallel eigensolvers in plane-wave density functional theory," *Computer Physics Communications*, vol. 187, pp. 98–105, 2015.

[18] S. Das, P. Motamarri, V. Gavini, B. Turcksin, Y. W. Li, and B. Leback, "Fast, scalable and accurate finite-element based *Ab Initio* calculations using mixed precision computing: 46 PFLOPS simulation of a metallic dislocation system," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '19)*. ACM, 2019, pp. 1–11.

[19] N. Kodali, K. Ramakrishnan, and P. Motamarri, "Residual-based Chebyshev filtered subspace iteration for Hermitian eigenvalue problems tolerant to inexact matrix-vector products," *Computer Physics Communications*, vol. 327, p. 110239, 2026.

[20] A. Sharma, A. Metere, P. Suryanarayana, L. Erlandson, E. Chow, and J. E. Pask, "GPU acceleration of local and semilocal density functional calculations in the SPARC electronic structure code," *The Journal of Chemical Physics*, vol. 158, no. 20, p. 204117, 2023.

[21] P. Giannozzi *et al.*, "Quantum ESPRESSO toward the exascale," *The Journal of Chemical Physics*, vol. 152, no. 15, p. 154105, 2020.

[22] X. Wu, D. Davidović, S. Achilles, and E. Di Napoli, "ChASE: A distributed hybrid CPU-GPU eigensolver for large-scale Hermitian eigenvalue problems," in *Proceedings of the Platform for Advanced Scientific Computing Conference (PASC '22)*. ACM, 2022, pp. 1–12.

[23] G. Panigrahi, N. Kodali, D. Panda, and P. Motamarri, "Fast hardware-aware matrix-free algorithms for higher-order finite-element discretized matrix multivector products on distributed systems," *Journal of Parallel and Distributed Computing*, vol. 192, p. 104925, 2024.

[24] I.-M. Lygatsika, M. Sarraute, L. Baguet, P. Kestener, and M. Torrent, "GPU acceleration of plane-wave density functional theory calculations in Abinit," 2026, arXiv:2604.11139.

[25] N. Kodali *et al.*, "Towards exascale fully relativistic pseudopotential density functional theory calculations with mixed-precision computation and compressed communication using residual-based subspace iteration," in *SC26: International Conference for High Performance Computing, Networking, Storage and Analysis*. Chicago, IL, USA: IEEE, 2026, to appear; arXiv:2605.30128.

[26] E. Anderson *et al.*, *LAPACK Users' Guide*, 3rd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1999.

[27] L. S. Blackford *et al.*, *ScaLAPACK Users' Guide*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 1997.

[28] J. J. Dongarra, D. C. Sorensen, and S. J. Hammarling, "Block reduction of matrices to condensed forms for eigenvalue computations," *Journal of Computational and Applied Mathematics*, vol. 27, no. 1–2, pp. 215–227, 1989.

[29] M. Lupo Pasini, B. Turcksin, W. Ge, and J.-L. Fattebert, "A parallel strategy for density functional theory computations on accelerated nodes," *Parallel Computing*, vol. 100, p. 102703, 2020.

[30] H. Wang *et al.*, "Improving tridiagonalization performance on GPU architectures," in *Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '25)*. ACM, 2025, pp. 469–480.

[31] J. Choi, J. J. Dongarra, and D. W. Walker, "The design of a parallel dense linear algebra software library: Reduction to Hessenberg, tridiagonal, and bidiagonal form," *Numerical Algorithms*, vol. 10, no. 2, pp. 379–399, 1995.

[32] C. H. Bischof, B. Lang, and X. Sun, "A framework for symmetric band reduction," *ACM Transactions on Mathematical Software*, vol. 26, no. 4, pp. 581–601, 2000.

[33] T. Auckenthaler *et al.*, "Parallel solution of partial symmetric eigenvalue problems from electronic structure calculations," *Parallel Computing*, vol. 37, no. 12, pp. 783–794, 2011.

[34] V. W.-z. Yu *et al.*, "ELSI — an open infrastructure for electronic structure solvers," *Computer Physics Communications*, vol. 256, p. 107459, 2020.

[35] TOP500, "TOP500 list — June 2026," https://www.top500.org/lists/top500/2026/06/, 2026, accessed 2026-11-12.

[36] T. Imamura, S. Yamada, and M. Machida, "Development of a high-performance eigensolver on a peta-scale next-generation supercomputer system," *Progress in Nuclear Science and Technology*, vol. 2, pp. 643–650, 2011.

[37] T. Fukaya and T. Imamura, "Performance evaluation of the EigenExa eigensolver on Oakleaf-FX: Tridiagonalization versus pentadiagonalization," in *2015 IEEE International Parallel and Distributed Processing Symposium Workshop (IPDPSW)*. IEEE, 2015, pp. 960–969.

[38] H. Wang *et al.*, "Rethinking back transformation in 2-stage eigenvalue decomposition on heterogeneous architectures," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '25)*. ACM, 2025, pp. 1830–1844.

[39] C. H. Bischof, B. Lang, and X. Sun, "Algorithm 807: The SBR toolbox—software for successive band reduction," *ACM Transactions on Mathematical Software*, vol. 26, no. 4, pp. 602–616, 2000.

[40] V. W.-z. Yu *et al.*, "GPU-acceleration of the ELPA2 distributed eigensolver for dense symmetric and Hermitian eigenproblems," *Computer Physics Communications*, vol. 262, p. 107808, 2021.

[41] A. Haidar, H. Ltaief, and J. Dongarra, "Parallel reduction to condensed forms for symmetric eigenvalue problems using aggregated fine-grained and memory-aware kernels," in *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC '11)*. ACM, 2011, pp. 1–11.

[42] A. Haidar, R. Solcà, M. Gates, S. Tomov, T. Schulthess, and J. Dongarra, "Leading edge hybrid multi-GPU algorithms for generalized eigenproblems in electronic structure calculations," in

Supercomputing (ISC 2013), ser. Lecture Notes in Computer Science. Springer, 2013, pp. 67–80.

[43] A. Haidar, S. Tomov, J. Dongarra, R. Solcà, and T. Schulthess, "A novel hybrid CPU–GPU generalized eigensolver for electronic structure calculations based on fine-grained memory aware tasks," *The International Journal of High Performance Computing Applications*, vol. 28, no. 2, pp. 196–209, 2014.

[44] H. Wang, L. Shi, Z. Duan, P. Wu, L. Guo, and S. Zhang, "Extracting the potential of emerging hardware accelerators for symmetric eigenvalue decomposition," 2024, arXiv:2410.02170.

[45] S. Zhang, R. Shah, H. Ootomo, R. Yokota, and P. Wu, "Fast symmetric eigenvalue decomposition via WY representation on tensor core," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP '23)*. ACM, 2023, pp. 301–312.

[46] H. Wang *et al.*, "Pipelined dense symmetric eigenvalue decomposition on multi-GPU architectures," 2025, arXiv:2511.16174.

[47] A. Marek *et al.*, "The ELPA library: scalable parallel eigenvalue solutions for electronic structure theory and computational science," *Journal of Physics: Condensed Matter*, vol. 26, no. 21, p. 213201, 2014.

[48] P. Kůs *et al.*, "Optimizations of the eigensolvers in the ELPA library," *Parallel Computing*, vol. 85, pp. 167–177, 2019.

[49] NVIDIA Corporation, "cuSOLVERMp," https://docs.nvidia.com/cuda/cusolvermp/, 2026, documentation; accessed 2026-11-12.

[50] R. Solcà, M. Simberg, R. Meli, A. Invernizzi, A. Reverdell, and J. Biddiscombe, "DLA-Future: A task-based linear algebra library which provides a GPU-enabled distributed eigensolver," in *Asynchronous Many-Task Systems and Applications (WAMTA 2024)*, ser. Lecture Notes in Computer Science, P. Diehl, J. Schuchart, P. Valero-Lara, and G. Bosilca, Eds., vol. 14626. Springer, Cham, 2024, pp. 135–141.

[51] M. Gates, J. Kurzak, A. Charara, A. YarKhan, and J. Dongarra, "SLATE: Design of a modern distributed and accelerated linear algebra library," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '19)*. Denver, CO: ACM, 2019, pp. 1–18.

[52] J. J. M. Cuppen, "A divide and conquer method for the symmetric tridiagonal eigenproblem," *Numerische Mathematik*, vol. 36, no. 2, pp. 177–195, 1980.

[53] J. J. Dongarra and D. C. Sorensen, "A fully parallel algorithm for the symmetric eigenvalue problem," *SIAM Journal on Scientific and Statistical Computing*, vol. 8, no. 2, pp. s139–s154, 1987.

[54] M. Gu and S. C. Eisenstat, "A divide-and-conquer algorithm for the symmetric tridiagonal eigenproblem," *SIAM Journal on Matrix Analysis and Applications*, vol. 16, no. 1, pp. 172–191, 1995.

[55] F. Tisseur and J. Dongarra, "A parallel divide and conquer algorithm for the symmetric eigenvalue problem on distributed memory architectures," *SIAM Journal on Scientific Computing*, vol. 20, no. 6, pp. 2223–2236, 1999.

[56] P. Arbenz, "Divide and conquer algorithms for the bandsymmetric eigenvalue problem," *Parallel Computing*, vol. 18, no. 10, pp. 1105–1128, 1992.

[57] J. Vogel, J. Xia, S. Cauley, and V. Balakrishnan, "Superfast divide-and-conquer method and perturbation analysis for structured eigenvalue solutions," *SIAM Journal on Scientific Computing*, vol. 38, no. 3, pp. A1358–A1382, 2016.

[58] W. N. Gansterer, R. C. Ward, and R. P. Muller, "An extension of the divide-and-conquer method for a class of symmetric block-tridiagonal eigenproblems," *ACM Transactions on Mathematical Software*, vol. 28, no. 1, pp. 45–58, 2002.

[59] W. N. Gansterer, R. C. Ward, R. P. Muller, and W. A. Goddard, "Computing approximate eigenpairs of symmetric block tridiagonal matrices," *SIAM Journal on Scientific Computing*, vol. 25, no. 1, pp. 65–85, 2003.

[60] L. Greengard and V. Rokhlin, "A fast algorithm for particle simulations," *Journal of Computational Physics*, vol. 73, no. 2, pp. 325–348, 1987.

[61] O. E. Livne and A. Brandt, "N roots of the secular equation in $O(N)$ operations," *SIAM Journal on Matrix Analysis and Applications*, vol. 24, no. 2, pp. 439–453, 2002.

[62] X. Ou and J. Xia, "SuperDC: Superfast divide-and-conquer eigenvalue decomposition with improved stability for rank-structured matrices," *SIAM Journal on Scientific Computing*, vol. 44, no. 5, pp. A3041–A3066, 2022.

[63] X. Liao, S. Li, Y. Lu, and J. E. Roman, "A parallel structured divide-and-conquer algorithm for symmetric tridiagonal eigenvalue problems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 32, no. 2, pp. 367–378, 2021.

[64] S. Li, X. Liao, Y. Lu, J. E. Roman, and X. Yue, "A parallel structured banded DC algorithm for symmetric eigenvalue problems," *CCF Transactions on High Performance Computing*, vol. 5, no. 2, pp. 116–128, 2023.

[65] G. Ballard, J. Demmel, O. Holtz, and O. Schwartz, "Minimizing communication in numerical linear algebra," *SIAM Journal on Matrix Analysis and Applications*, vol. 32, no. 3, pp. 866–901, 2011.

[66] T. Fukaya, Y. Nakatsukasa, Y. Yanagisawa, and Y. Yamamoto, "CholeskyQR2: A simple and communication-avoiding algorithm for computing a tall-skinny QR factorization on a large-scale parallel system," in *2014 5th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA)*. IEEE, 2014, pp. 31–38.

[67] Y. Yamamoto, Y. Nakatsukasa, Y. Yanagisawa, and T. Fukaya, "Roundoff error analysis of the CholeskyQR2 algorithm," *Electronic Transactions on Numerical Analysis*, vol. 44, pp. 306–326, 2015.

[68] G. Ballard, J. Demmel, L. Grigori, M. Jacquelin, H. D. Nguyen, and E. Solomonik, "Reconstructing Householder vectors from tall-skinny QR," in *2014 IEEE 28th International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2014, pp. 1159–1170.

[69] G. Ballard, J. Demmel, L. Grigori, M. Jacquelin, N. Knight, and H. D. Nguyen, "Reconstructing Householder vectors from tall-skinny QR," *Journal of Parallel and Distributed Computing*, vol. 85, pp. 3–31, 2015.

[70] R. Schreiber and C. Van Loan, "A storage-efficient WY representation for products of Householder transformations," *SIAM Journal on Scientific and Statistical Computing*, vol. 10, no. 1, pp. 53–57, 1989.

[71] W. Fong and E. Darve, "The black-box fast multipole method," *Journal of Computational Physics*, vol. 228, no. 23, pp. 8712–8725, 2009.

[72] C. Puglisi, "Modification of the Householder method based on the compact WY representation," *SIAM Journal on Scientific and Statistical Computing*, vol. 13, no. 3, pp. 723–726, 1992.

[73] T. Joffrain, T. M. Low, E. S. Quintana-Ortí, R. van de Geijn, and F. G. Van Zee, "Accumulating Householder transformations, revisited," *ACM Transactions on Mathematical Software*, vol. 32, no. 2, pp. 169–179, 2006.

[74] R. C. Whaley, A. Petitet, and J. J. Dongarra, "Automated empirical optimizations of software and the ATLAS project," *Parallel Computing*, vol. 27, no. 1–2, pp. 3–35, 2001.

[75] J. Lee, "New supercomputer Hangang is 24 times faster than previous model," Korea.net, https://www.korea.net/NewsFocus/Sci-Tech/view?articleId=302150, Nov. 2026, published 3 November 2026; accessed 2026-11-12.

[76] S. Atchley *et al.*, "Frontier: Exploring exascale," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '23)*. ACM, 2023, pp. 1–16.

[77] W. E. Allcock *et al.*, "Aurora: Architecting Argonne's first exascale supercomputer for accelerated scientific discovery," 2025, arXiv:2509.08207.

[78] NVIDIA Corporation, "NVIDIA GH200 Grace Hopper Superchip datasheet," https://dam-cdn.nvd.orangelogic.com/AssetLink/tkno03fa645gb7f33w60rf200d330420.pdf, 2025, document 3773000, March 2025; accessed 2026-11-12.

[79] Oak Ridge Leadership Computing Facility, "Frontier user guide," https://docs.olcf.ornl.gov/systems/frontier_user_guide.html, accessed 2026-11-12.

[80] Argonne Leadership Computing Facility, "Aurora," https://www.alcf.anl.gov/aurora, system specifications; accessed 2026-11-12.

[81] Intel Corporation, "Intel Data Center GPU Max 1550 – product specifications," https://www.intel.com/content/www/us/en/products/sku/232873/intel-data-center-gpu-max-1550/specifications.html, accessed 2026-11-12.

[82] DFT-FE developers, "DFT-FE benchmarks," https://github.com/dftfeDevelopers/dftfe-benchmarks, 2026, case performanceBenchmarks/DFTFEv1.0/Summit/GroundStateCalculations/MinWallTimes/BCCMoSuperCells/8x8x8Vac; accessed 2026-11-12.